

Real Time Neuron Labeling in AI Models

From neuron probes to NeuronCards, with a runnable toy demo

Author: Machiavalli AI research assistant
Included: toy AI neuron labeling demo (NumPy)

Abstract

Labeling each neuron in a modern neural network is harder than it sounds because many neurons are polysemantic and their apparent function depends on context. Nevertheless, it is possible to produce a useful live map by changing the output from a single word label to a structured NeuronCard: a small set of candidate concepts, an uncertainty distribution, and causal test results from ablation or activation patching. This paper describes a practical loop for real time labeling during inference or training: controlled stimulus sweeps, streaming statistics, targeted probes, and causal interventions. A small demo is included that recovers concept labels for hidden units in a toy model.

1. What you mean by label each neuron

A useful definition is: for each neuron or unit, produce a compact description that predicts when it activates, what features it encodes, and what behaviors it causally affects. Because most units are not perfectly monosemantic, the label should be a ranked set of hypotheses rather than a single claim. Network Dissection explicitly frames interpretability as aligning individual units to semantic concepts, which is the closest thing to an operational definition of neuron labeling in vision models.

2. Why real time

Real time labeling is valuable when you want to debug a model while it runs, monitor distribution shift, or drive an active probing loop that selects the next inputs to reduce uncertainty. The same idea appears in mechanistic interpretability techniques that treat model internals as causal variables rather than just correlates. Activation patching work emphasizes subtleties and provides guidance on how to interpret swaps of internal activations.

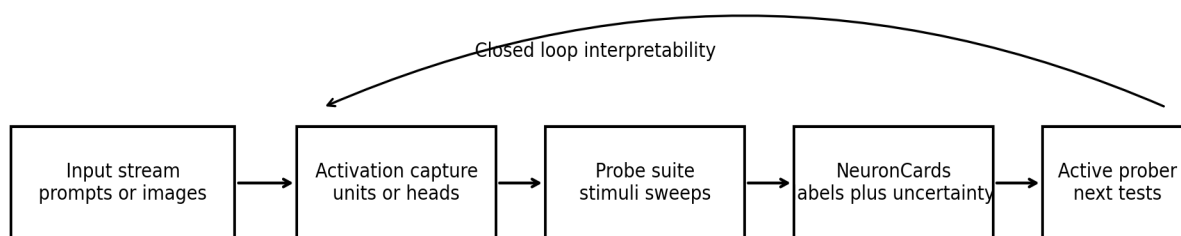


Figure 1. A practical real time neuron labeling loop for AI models.

3. Core techniques for neuron labeling

A robust NeuronCard is assembled from three complementary evidence types. First, correlational probes: measure how activation changes across controlled input factors or concept labeled datasets, as in Network Dissection. Second, generative visualization: synthesize inputs that maximize a unit activation, as in feature visualization work. Third, causal tests: ablate the unit or patch activations between runs to measure effect on the output, using activation patching style tools.

4. Better units than neurons

If individual neurons are strongly polysemantic, change the unit of analysis. Dictionary learning over activations can recover sparse features that behave more like semantic atoms than raw neurons. This line of work argues that features, not single neurons, are often the right objects to label. Real time labeling can still proceed, but your NeuronCards should target discovered features as well as neurons.

AI neuron label snapshot from the toy demo

unit	mean_act	label_single	conf_single	label_pair	conf_pair
0	3.02	c0	0.97	c0+c1	0.0
1	3.01	c1	0.97	c2+c3	0.03
8	4.52	c0	0.49	c0+c1	0.98
9	4.49	c3	0.49	c2+c3	0.98

Figure 2. Example label snapshot from the included toy AI demo.

5. Included demo

The demo is intentionally small and uses only NumPy. It defines a toy model whose hidden units implement known concept detectors and simple concept interactions. A streaming labeler estimates for each hidden unit which concept best explains its activation by comparing activation means conditioned on concept presence. This reproduces the control loop: controlled stimuli, streaming stats, and per unit label vectors.

Run commands (Windows PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python src_ai\run_ai_demo.py
```

6. Scaling notes

Scaling to large transformers requires automation and careful measurement. You typically do not label every unit with a bespoke experiment. Instead you maintain a continuously updated library of probes, compute unit concept scores, and prioritize causal tests for the most behavior relevant units. The prober chooses which tests to run next based on uncertainty, budget, and value of information for your current goal.

References

1. Bau D, Zhou B, Khosla A, Oliva A, Torralba A. Network Dissection: Quantifying Interpretability of Deep Visual Representations. CVPR. 2017. arXiv:1704.05796.
2. Olah C, Mordvintsev A, Schubert L. Feature Visualization. Distill. 2017. doi:10.23915/distill.00007.
3. Transformer Circuits. Decomposing Language Models With Dictionary Learning. 2023.
4. How to use and interpret activation patching. 2024. arXiv:2404.15255.